Lab 10

Polymorphism, Interfaces & Abstraction

Objectives: At the end of this lab session the student should be able to:

- Apply polymorphism in Java application development.
- Declare and implement Java interfaces & abstract classes.

Part 1: Problem Solving Exercises (Time: 75 minutes)

Exercise 1

Create a package named excercise1, class Person, and enumeration Gender.

```
import java.time.LocalDate;

public class Person {
	private String name;
	private LocalDate birthDate;
	private Gender gender;
	public Person() {}
	public Person(String name, LocalDate birthDate, Gender gender) {
		this.name = name;
		this.birthDate = birthDate;
		this.gender = gender;
	}
	public String getName() {
		return name;
	}
	public void setName(String name) {
		this.name = name;
	}
	public LocalDate getBirthDate() {
		return birthDate;
	}
	public void setBirthDate(LocalDate birthDate) {
		this.birthDate = birthDate;
	}
	public Gender getGender() {
		return gender;
	}
	public void setGender(Gender gender) {
		this.gender = gender;
	}
}

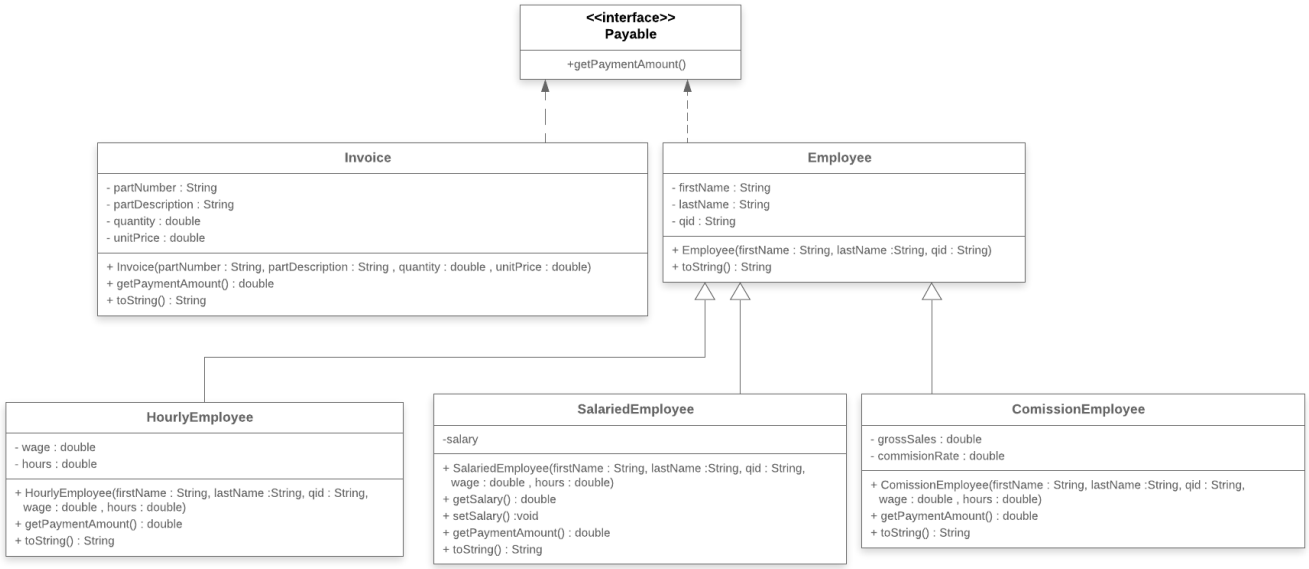
public enum Gender {
	MALE, FEMALE;
}

import java.time.LocalDate;
public class PersonTest {
	public static void main(String[] args) {
```

```
                // TODO Auto-generated method stub
                LocalDate date = LocalDate.of(2001, 1, 15);
                Person p1 = new Person("Mona", date, Gender.FEMALE);
                Person p2 = new Person("Ali", LocalDate.of(2002, 11, 20), Gender.MALE);
                if (p1.getBirthDate().isBefore(p2.getBirthDate()))
                        System.out.println("P1 is older than P2");
                else
                        System.out.println("P2 is older than P1");
        }
}
```

Exercise 2

Create a package named excercise2 to place all the classes of this exercise. Implement the following class hierarchy:



- Test your implementation using the main method.

```
import java.util.ArrayList;

public class PayableTest {
        public static void main(String[] args) {
                // TODO Auto-generated method stub
                // create payable list
          ArrayList<Payable>  payables= new ArrayList<Payable>();

            // populate list with objects that implement Payable
            payables.add(new Invoice( "01 234", "Textbook", 2, 375.00 ));
            payables.add(new Invoice( "56 789", "USB Disk", 3, 179.95 ));
            payables.add(new SalariedEmployee( "Ahmed", "Ali", "111-11-1111", 15 000.00 ));
            payables.add(new HourlyEmployee( "Fatima", "Saleh", "222-22-2222", 160.75, 40 ));
            payables.add(new CommissionEmployee( "Samir", "Sami", "333-33-3333", 100 000, .06 ));

            System.out.println("Invoices and Employees processed polymorphically:\n" );
            // generically process each element in array payableObjects
            for ( Payable payable : payables )
            {
              // output currentPayable and its appropriate payment amount
              System.out.printf( "ObjectType: %s - PaymentAmount = QR %.2f\n",
```

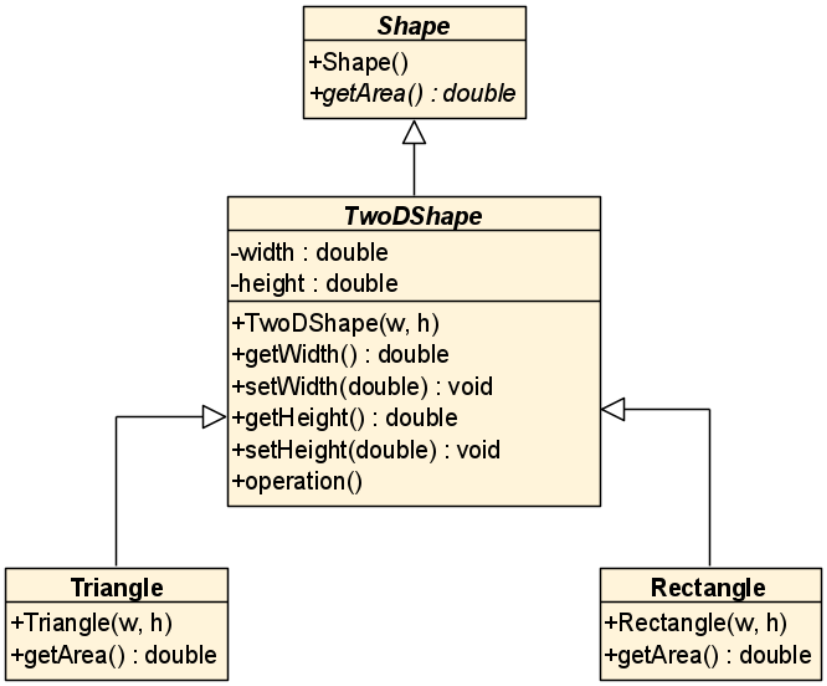
```
                        payable.getClass().getSimpleName(),
                        payable.getPaymentAmount() );
                //If SalariedEmployee then increase the salary by 10%
                if ( payable instanceof SalariedEmployee)
                {
                  // downcast Payable reference to
                  // BasePlusCommissionEmployee reference
                     SalariedEmployee salariedEmployee = ( SalariedEmployee ) payable;
                  double oldBaseSalary = salariedEmployee.getSalary();
                  salariedEmployee.setSalary(oldBaseSalary * 1.1);
                  System.out.printf( "New salary with 10%% increase is: QR %,.2f\n",
                              salariedEmployee.getSalary() );
                }
              }
          }
}
```

Exercise 3

Consider the following class diagram, implement all four classes represented in the hierarchy.

Classes Shape and TwoDShape are abstract classes.

Triangle area = 0.5 * height * width Rectangle area = height * width



Part 2: Additional Practice Exercises (Time: 75 minutes)

Exercise 4

Expert Cleaning Co. provides cleaning services to residents of the Pearl Qatar. You are requested to build an app to allow them to manage the cleaning services provided and the fees to be collected from residents.

a) The class diagram of the application is shown in the figure below (Figure1 is based on the Exercises of Lab 9).

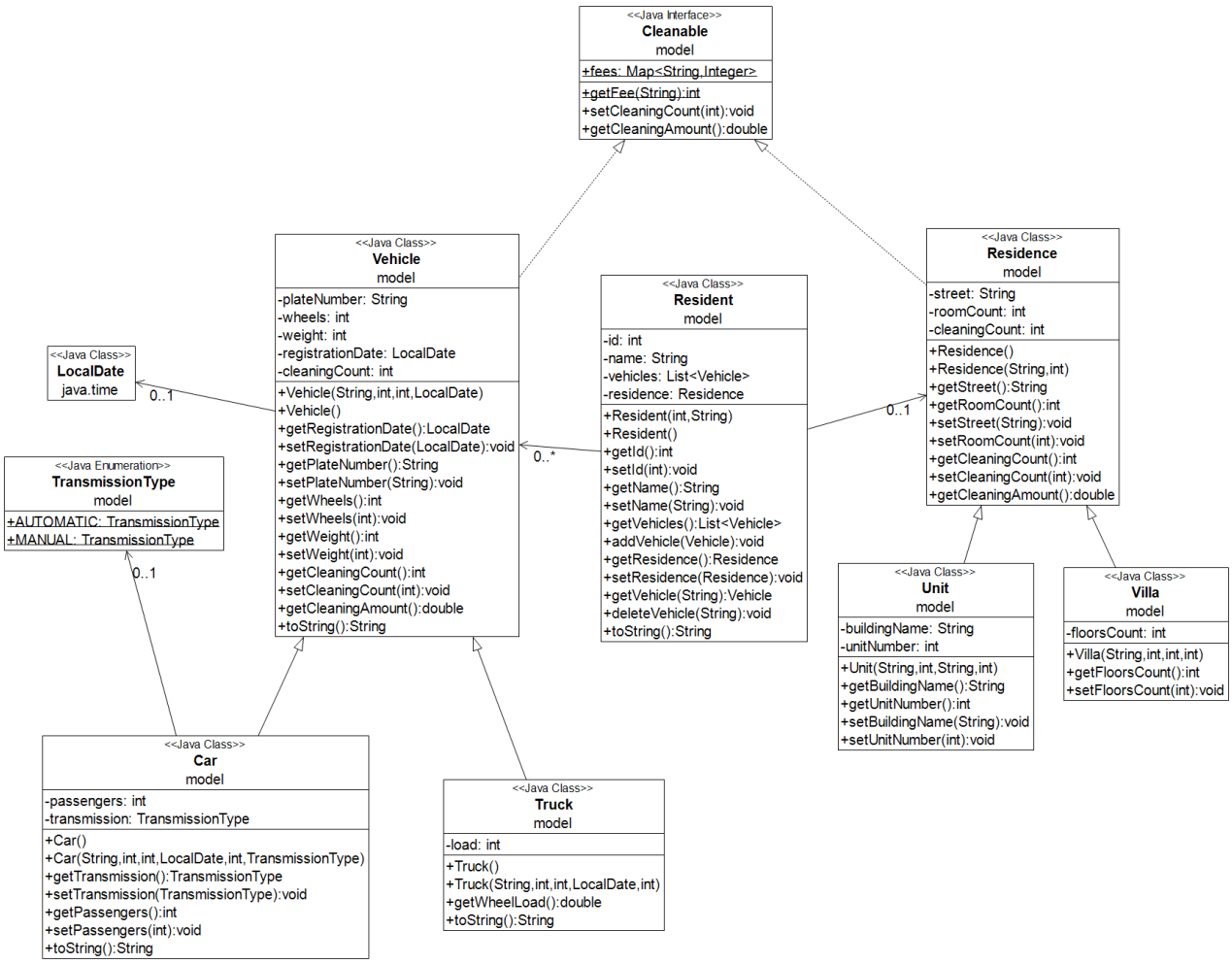


Figure 1 - CleaningCoApp class diagram (A larger diagram is given at the end)

You are required to modify the `Vehicle`, `Car`, `Truck` and `Resident` classes to complete the implementation of the above inheritance hierarchy. Create the TransmissionType as enum. Import `java.time.LocalDate`. The methods' descriptions are given below.

Cleanable Interface:

- Use the following Map values for *fees*:

  Map.*of*("Car", 10, "Truck", 15, "Unit", 25, "Villa", 50);

Vehicle class:

- Implements the Cleanable interface.
- Modify the parameterized constructor so that it takes five arguments (add `plateNumber`, `cleaningCount` and `registrationDate`).

- Add the setters and getters for the new fields.
- Implement the interface methods. The setCleaningCount(int) is just a setter. The getFee(String) returns the fee from the ***map*** using the string. As for the getCleaningAmount(), this method should return the product of the cleaningCount by the cleaning fee of the *class type.* (Hint: call getFee(String) method of the interface)
- Modify the `toString()`so that it includes `plateNumber` and `registrationDate` but only if the `plateNumber` value is not null. Otherwise return "".

Car class:

- Modify the parameterized constructor to take the six arguments (see class diagram).

Truck class:

- Modify the parameterized constructor to take the six arguments (see the class diagram).

Residence class:

- Implement the class as shown in the class diagram. This class implements Cleanable.
- Implement the interface methods in a similar fashion to what you did in class Vehicle.
- Override toString() method to return a formatted string representation of the instance.

Unit class:

- Implement the class as shown in the class diagram. This class extends Residence.
- Override toString() method to return a formatted string representation of a Unit instance.

Villa class:

- Implement the class as shown in the class diagram. This class extends Residence.
- Override toString() method to return a formatted string representation of a Villa instance.

Resident class:

- Rename the Person class (from Lab09) to be Resident.
- Add a `residence` attribute and generate its setter and getter.
- Add a `vehicles` list as a private attribute and create its getter method (a setter is not required as it is meaningless).
- Add the `addVehicle()` method which adds a Vehicle parameter to the vehicles list. It should check that the parameter value is not null and that it does not already exist in the vehicles list.
- Add the following methods:

```
public Vehicle getVehicle(String plateNumber) {
        // searches for a vehicle with plateNumber and returns it or returns null if not found.
}

public void deleteVehicle(String plateNumber) {
        // searches for a vehicle with plateNumber and deletes it if found.
}
```